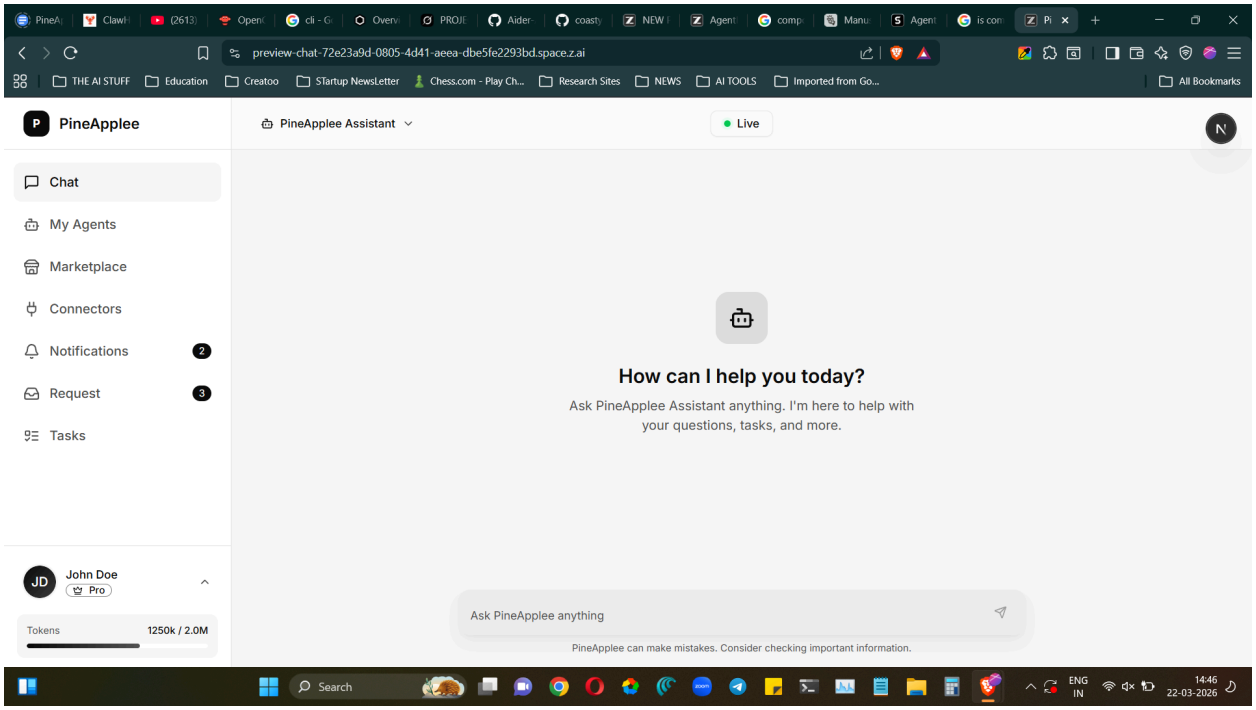

SYSTEM ARCHITECTURE (CHAT WINDOW)

1. Core Agent Framework (OpenClaw)

This is the backbone of the system where all agents operate. OpenClaw will manage how agents think, act, and execute tasks.

- **What it does:**
  - Runs multiple agents simultaneously
  - Handles task execution logic
  - Allows agents to perform background operations
- **How it works:**
  - Each user action triggers an agent process
  - Agents break tasks into smaller steps and execute them
  - Supports modular structure → agents can be extended with skills (coming soon)

2. LLM Infrastructure (Self-Hosted)

All AI models are hosted internally instead of using external APIs.

- **What it does:**
  - Provides intelligence to agents
  - Handles reasoning, generation, and decision-making

- **Models Used:**
    - Qwen, MiniMax, DeepSeek R1, Gemma 4 (Google's new model)
- **How it works:**
    - Requests from agents are routed to selected LLM
    - Based on task type, model selection can be optimized later
    - Internal hosting ensures:
        - Cost control
        - Data privacy
        - Faster response (if optimized)

SECURITY LAYERS

1. Multi-Layer Security System

Security is critical because agents can perform actions on behalf of users.

- **What it does:**
    - Prevents harmful or unauthorized actions
    - Ensures controlled execution of tasks
- **How it works:**
    - Every action goes through validation layers
    - Risky actions are flagged before execution

(WHAT ALL SECURITY LAYERS ARE ADDED WILL BE DISCUSSED SOON)

2. Human Intervention Loop

This is a **manual approval system** for sensitive operations.

- **What it does:**
    - Stops agents from executing critical actions without user consent
- **Sensitive Actions:**
    - Payments, purchases, sending data, pushing actions etc
- **How it works:**
    - Agent initiates action → request generated
    - User receives approval request in "Requests" section
    - Only after approval → action executes

3. Fail-Safe + Logging

Ensures traceability and rollback understanding.

- **What it does:**
    - Tracks every action taken by agent and user
    - Helps debug issues and audit behavior

- **How it works:**
    - Logs stored in structured format
    - Accessible via “Logs” section (terminal style)

CHAT SYSTEM (CORE PRODUCT)

1. Multi-Agent Chat

This is the main interface where users interact with agents.

- **What it does:**
    - Allows chatting with multiple agents in one interface
    - Agents perform tasks while conversation continues
- **How it works:**
    - Each agent runs as a parallel process
    - Chat UI streams responses while backend executes tasks

2. Usage Limits (Free Tier)

Controls system load and monetization.

- **What it does:**
    - Restricts free users to limited usage
- **How it works:**
    - Daily cap (5–10 tasks)
    - Task length restrictions applied
    - System checks limits before execution

3. Queue Management System

Handles system overload situations.

- **What it does:**
    - Prevents system crash under heavy load
- **How it works:**
    - Tasks are queued when capacity is full
    - User sees “In Queue” message
    - Tasks processed FIFO (can upgrade to priority queue later)

4. Token System

This is the core billing + usage tracking system.

- **What it does:**
    - Tracks AI usage based on input/output tokens

- **How it works:**
    - Each request consumes tokens
    - Tokens deducted in real-time
    - UI shows usage via progress bar

WEB APP INTERFACE

Overall Experience

The UI is designed to feel like premium AI tools (ChatGPT/Grok style).

- **What it does:**
    - Provides a smooth and intuitive interaction layer
- **How it works:**
    - Component-based frontend (React recommended)
    - Real-time updates (WebSockets for chat/tasks)
    - Theme switching (Dark/Light)

SIDEBAR MODULES

1. Chat

Primary working area of the platform.

- **What it does:**
    - Allows user to interact with selected agent(s)
- **How it works:**
    - Chat messages → sent to backend → processed by agent → response streamed back

2. My Agents

This is the user's personal agent management section.

- **What it does:**
    - Displays all owned agents
    - Allows customization via skills (coming soon)
- **How it works:**
    - Each agent stored as an entity with metadata
    - "Add Skills" opens modal → updates agent config (coming soon)
    - Skills enhance agent capability (APIs/tools/tasks)

3. Marketplace

Handles monetization and expansion.

A. Subscriptions

- **What it does:**
  - Controls access level (agents + tokens)
- **How it works:**
  - Plan selection updates backend entitlements
  - Limits enforced at runtime

B. Token Packs

- **What it does:**
  - Enables additional usage purchase
- **How it works:**
  - Tokens added to wallet after payment
  - Deducted per request

B. Choose Agents

- **What it does:**
  - User can choose agents based on their subscription model
- **How it works:**
  - User choose the subscription
  - Based on chosen subscription they can choose any agents from the marketplace and use it

4. Connectors (MCP) (COMING SOON)

External integrations layer.

- **What it does:**
  - Connects agents to third-party tools
- **How it works:**
  - Uses Composio APIs
  - OAuth-based authentication
  - Permissions stored per user

5. Notifications

Basic alert system.

- **What it does:**

        - Informs users about updates/events
- **How it works:**
    - Triggered by backend events
    - Stored and displayed in list format

6. Requests (Approval Engine)

Critical security UI.

- **What it does:**
    - Displays pending approval actions
- **How it works:**
    - Each sensitive action creates request entry
    - User action determines execution
    - Reject triggers confirmation step
    - Until and unless the user wont approve, the agent wont be moving forward with the task that has been assigned to it.

7. Tasks

Execution tracking dashboard.

- **What it does:**
    - Shows all agent activities
- **How it works:**
    - Each task logged with:
        - Status
        - Time
        - Result
    - Helps users monitor automation

PROFILE & SYSTEM PANEL

1. Profile

- Displays user identity (read-only initially)

2. Logs

- **What it does:**
    - Full transparency of system activity
- **How it works:**
    - Terminal-style UI
    - Stores input-output pairs

3. Live Status

- **What it does:**
  - Shows real-time system health
- **How it works:**
  - Pulls metrics like:
    - Latency
    - Server load
    - Agent responsiveness

4. Billing

- **What it does:**
  - Tracks all financial activity
- **How it works:**
  - Stores:
    - Transactions
    - Subscription details
    - Token purchases

CHAT INPUT SYSTEM

This is a **key differentiator UI component**.

- **What it does:**
  - Makes interaction flexible and powerful
- **Features & Working:**
  - File Upload → Sends file to backend parser
  - Drag & Drop → Same as upload
  - Voice Input → Converts speech to text → sent as prompt (coming soon)

TOKEN TRACKING UI

- **What it does:**
  - Shows consumption visibility
- **How it works:**
  - Progress bar linked to backend token counter
  - Updates in real-time after each request

AGENT DEPLOYMENT

- **What it does:**
  - Activates selected agent for usage

- **How it works:**
    - User selects agent → clicks deploy
    - Confirmation popup prevents accidental deployment
    - Once deployed → agent becomes active (non-reversible)

UI/UX SYSTEM

Design Philosophy

- Premium, minimal, futuristic

Behavior

- Hover animations on icons
- Smooth transitions
- Micro-interactions for feedback

Implementation

- Use:
    - Tailwind / modern UI frameworks
    - Animation libraries (Framer Motion)

FUTURE IMPROVEMENTS

- More fast servers with less latency and high value output
- Priority execution for paid users
- Smart LLM routing
- Third-party agent marketplace
- Advanced analytics dashboard

ADMIN PANEL

WORKFLOW

SCREEN 1: USER ONBOARDING FLOW

[User Lands on Website]

↓

Clicks "Get Started"

↓

[Chat Interface Opens - Unauthenticated State]

↓

System Check: Is User Logged In?

↓

NO → Show Auth Options:

→ Continue with Google

→ Login/Signup with Email

↓

User Completes Authentication

↓

Backend:

- Validate credentials
- Create/Fetch User Profile

↓

[User Redirected to Chat Dashboard]

↓

Default State:

- Free Tier Activated
- 1 Default Agent Assigned

SCREEN 2: FREE USER FLOW

[User Enters Dashboard]

↓

Top Notification:

"You are using Free Version – Upgrade to unlock more features"

↓

System Assigns:

- 1 Default Agent (Non-changeable)

↓

User Sends Task

↓

System Checks:

- Daily Task Limit (Max: 5)
- Task Complexity

↓

IF Within Limit:

→ Task Executed

↓

IF Limit Exceeded:

→ Show: "Daily task limit reached"

↓

IF Task Too Large:

→ Restrict / Ask to Upgrade

↓

Restricted Access:

- ❌ Token Section (Locked) (it would be grey, user can view but cannot select anything)
- ❌ Agent Selection (Locked) (it would be grey, user can view but cannot select anything)

(under marketplace section we have 3 options: Subscription, Tokens, Agents Selection)

SCREEN 3: SUBSCRIPTION FLOW (PAID USER)

[User Navigates → Marketplace → Subscription]

↓

Display Plans:

- Free
- ₹999 / ₹2999 / ₹9999 / Custom

↓

User Selects Plan

↓

Clicks "Subscribe"

↓

Redirect → Razorpay Payment Gateway

↓

Payment Processing

↓

IF Payment SUCCESS:

→ Razorpay Callback → Backend Verification

→ Update User Plan

→ Unlock:

- Agent Selection
- Token Section

↓

→ Show Success Popup

↓

→ Redirect to Dashboard

IF Payment FAILED:

→ Show Error Message

→ Stay on Subscription Page

(When the user is in free tier, they can see the tokens page and agents page, they can only see those pages but cannot select or perform any activity there, we want the user that they can easily see the agents what kind of agents we have, and tokens we have, so there is nothing in that, just grey and view only)

SCREEN 4: AGENT SELECTION FLOW

[User Navigates → Marketplace → Agents]

↓

System Check:

Is Subscription Active?

↓

NO:

→ Show: "Please activate subscription to choose agents"

→ Block Selection

YES:

↓

Fetch Plan Limits:

- Max Agents Allowed (Example, basic plan allows only 3 agents to choose)

↓

User Selects Agent(s)

↓

IF Within Limit:

→ Can select from the pool of agents

→ Available in Chat Dropdown

IF Exceeds Limit:

→ Show: "Agent limit exceeded as per your plan"

Then Choose Deploy agents – > a pop up appear which says a caution message which says this task cannot be reversed, please be cautious and choose the agents carefully (even in the pop up we can share the list of agents that they have chosen, like we see in the ecommerce page before checkout)

SCREEN 5: TOKEN PURCHASE FLOW

[User Navigates → Marketplace → Tokens]

↓

System Check:

Is Subscription Active?

↓

NO:

→ Lock Page

→ Show: "Upgrade to access tokens"

YES:

↓

Display Token Packs

↓

User Selects Pack

↓

Redirect → Razorpay

↓

Payment Processing

↓

IF SUCCESS:

→ Add Tokens to Wallet

→ Update Token Progress Bar

→ Show Confirmation

IF FAILED:

→ Show Error Message

SCREEN 6: CHAT & TASK EXECUTION FLOW

[User Sends Prompt in Chat]

↓

System Checks:

- Token Balance
- Task Limits

↓

IF Valid:

→ Send Task to Agent

↓

Agent Starts Processing

↓

Check Task Type:

↓

[Normal Task]

→ Execute Directly

→ Return Response

↓

[Sensitive Task]

→ Pause Execution

→ Send Request to Approval System

SCREEN 7: REQUEST APPROVAL FLOW

[Agent Triggers Sensitive Action]

↓

Create Request Entry

↓

Send to "Requests Section"

↓

User Views Request:

- Approve
- Reject

↓

IF Approve:

→ Resume Task Execution

→ Complete Task

IF Reject:

→ Show Confirmation Popup:

"Are you sure?"

↓

YES → Cancel Task

NO → Return to Pending State

SCREEN 8: TASK TRACKING FLOW

[Every Task Executed]

↓

Log Entry Created:

- Task Type
- Status (Completed / Pending / Rejected)
- Timestamp
- Agent Used

↓

Store in Database

↓

Display in "Tasks Section"

↓

User Can View:

- Task History
- Status Breakdown

SCREEN 9: NOTIFICATION FLOW

[Event Triggered]

- System Event
- Admin Action

↓

Create Notification Entry

↓

Store in Database

↓

Display in Notification Panel

↓

User Actions:

- View Notification
- Mark All as Read

SCREEN 10: CHAT MEMORY FLOW

[User Sends Chat]

↓

Store:

- Message
- Agent Used
- Timestamp

↓

Link to User ID

↓

Save in Database

↓

User Can Access Chat History Anytime

SCREEN 11: BILLING FLOW (RAZORPAY)

[User Initiates Payment]

- Subscription OR Tokens

↓

Redirect → Razorpay

↓

Payment Processing

↓

IF SUCCESS:

→ Razorpay Callback

→ Backend Verifies Payment

→ Update:

- Subscription Status OR
- Token Balance

↓

→ Store Transaction:

- Transaction ID
- Amount
- Status

IF FAILED:

→ Show Failure Message

SCREEN 12: LOGGING FLOW

[Every Interaction]

↓

Capture:

- User Input (Prompt)
- Agent Output (1-line summary)

↓

Store in Logs

↓

Display in "Logs Section"

↓

Format:

- Terminal Style View

(this log section would be non editable from any side, it would be visible to user as well as to our admin panel)

FINAL MASTER FLOW (COMPLETE SYSTEM)

User → Auth (Google / Email)

↓

Free Tier Activated

↓

User Uses Chat

↓

Upgrade → Subscription

↓

Unlock Agents + Tokens

↓

Chat → Task → Agent

↓

IF Sensitive → Approval Loop

↓

Execution → Result

↓

Store → Tasks / Logs / Memory